CS 6210 Fall 2025 Test 2

(120 min Canvas Quiz)

[Total Points: 80]

Table of Contents

Distributed Systems [24 points] 1
  Lamport's Logical Clock [8 points] 2
  Lamport's ME Algorithm [6 points] 4
  RPC Latency Limits [6 points] 5
  Active Networks [4 points] 6
Distributed Objects and Middleware [16 points] 6
  Spring OS [11 points] 7
  EJB [2 points] 8
  Java RMI [3 points] 8
Distributed Subsystems [24 points] 9
  Global Memory Systems [10 points] 9
  Distributed Shared Memory [10 points] 10
  Distributed File Systems [4 points] 12
HIDDEN - Potpourri [16 points] 13

Distributed Systems [24 points]

Lamport's Logical Clock [8 points]

1. [3 points] Students are designing a multiplayer game where players send updates to a shared game state. Each player's move <playerID, action> is broadcast to all other players.
Each player can respond to a move by performing a counteraction that is also broadcast.

They propose using Lamport's logical clocks to order all actions, so that each player's screen displays actions in the exact order they were generated.

Assume:
No message loss.
Reliable but variable latency network.
In order delivery between any two players.

Will Lamport's logical clocks ensure that a counteraction to a move always appears after the corresponding original move for every player? State yes or no and provide reasoning for the same.

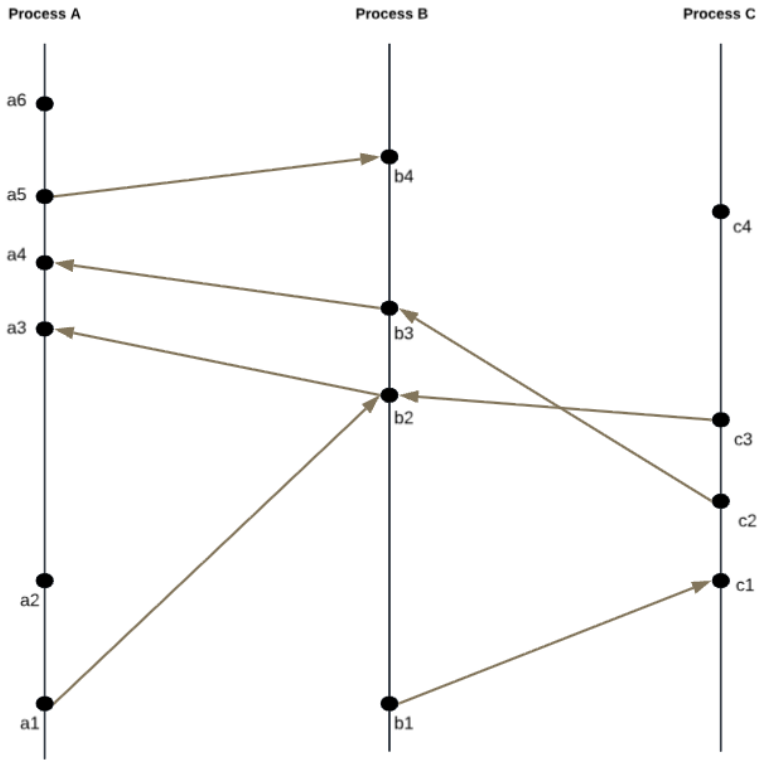
Ans:

- [+1] No.
- [+2] Lamport clocks maintain causal ordering but not real-time consistency across all nodes. If Player A performs a move and Player B sends a counteraction, another player C might receive B's counteraction before A's move due to message delay. Hence, Lamport timestamps cannot ensure that all participants view actions in exact generation order.

2. [5 points]

For all questions in this section, explain your reasoning in one or two sentences. Correct justification is required to receive credit.

Consider the diagram below. The vertical direction represents time, later times being higher than earlier ones. The dots denote events, the vertical lines denote processes, and the arrows denote messages. (Lamport, 1978)



Consider a partial ordering of the events shown.

a) [1 point] Did b3 happen before a6? Explain your reasoning.

Ans: [+1] Yes. b3->a4->a5->a6

b) [1 point] Did c3 happen before a4? Explain your reasoning.

Ans: [+1] Yes. c3->b2->a3->a4

c) [1 point] Did a2 happen before c1? Explain your reasoning.

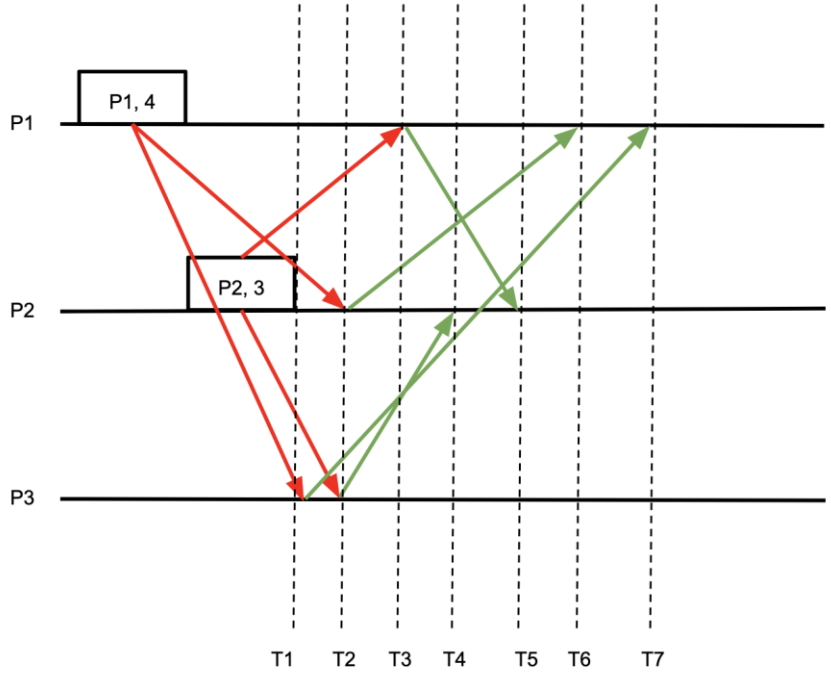
Ans: [+1] No. No causal relationship between these two events, they will be concurrent.

d) [2 points] In the diagram with Processes A, B, and C, Lamport's logical clocks establish a partial ordering of events. How can this ordering be extended to a total ordering of all events in the system?

Ans: To obtain a total order, if two events have the same logical timestamp, any arbitrary but consistent rule could be used to break the tie (e.g., using process IDs A < B < C).

Lamport’s ME Algorithm [6 points]

1. [6 points]



Consider a distributed system with 3 nodes (P1, P2, P3) implementing Lamport's mutual exclusion algorithm, as shown in the Figure. The boxes represent requests made by a node at a particular timestamp. For example, P1,4 represents a request for the resource made by node P1 at timestamp 4. The red arrows represent requests, and green arrows represent the corresponding acknowledgment messages.

a) [2 points] Which node gets access to the resource first and at which point in time (T1-T7)? Justify your answer to get any credit.

Ans:

- [+1] P2 gets access to the resource first as it made the request with the earliest timestamp.
- [+1] T4: To get access to the resource, P2 must receive messages with timestamps greater than its own request from all other nodes. At T2, it receives a request with a greater timestamp from P1. At T4, it receives an acknowledgment from P3. Hence, it gets access to the resource at T4.

b) [2 points] The algorithm assumes that any message sent by a process is eventually delivered to its intended recipient. Mention one problem that could occur if this assumption is violated.

Ans:

- [+2] Loss of a request message: Suppose process P1 sends a request to P2, but the message is lost. If P2 later sends its own request to P1 and receives an acknowledgment, it might enter the critical section, unaware that P1 is still using the shared resource, violating mutual exclusion.
  OR
  Loss of an acknowledgment message: If an acknowledgment is lost, the requesting process may keep waiting indefinitely, causing a deadlock.

c) [2 points] Suggest a simple method to prevent the algorithm from failing due to message loss.

Ans:

- [+2] Use a message acknowledgment protocol: Upon receiving a message, a node sends an acknowledgment message back to the sender. If the sender does not receive this acknowledgment message within a particular amount of time, it assumes that the message was lost and resends it. This ensures that all messages are eventually delivered.

- OR Attach sequence numbers to each message. If a process receives messages out of order or if there is a gap in the sequence, it can request the sender to retransmit the missing message. OR Any other alternate solution, if correct, will be accepted.

RPC Latency Limits [6 points]

1. [6 points] A legacy low-latency RPC system for a distributed operating system operates over 100 Gbps Ethernet hardware that occasionally exhibits packet corruption, even when Ethernet CRC checks pass. To guard against such errors, the system currently uses software-based UDP checksums, which add about 180 µs per RPC, contributing roughly 7% of the total latency for a minimal RPC call.

An improved integrity verification mechanism is being considered to reduce latency while maintaining correctness. The following 2 approaches are being considered -

- Hardware-only -> Use NIC checksum offload, no software checksums
- Hardware Software Combined -> Use NIC checksum offload together with an additional lightweight application-level checksum in the RPC protocol.

a) [2 points] If you have two machines connected via 400 Gbps Ethernet in a LAN, which check-sum strategy would you use? Justify your answer. [2 points]

Answer – Hardware-only checksums [+1]

LAN is more reliable than WAN so leverage everything that the hardware gives you [+1]

b) [2 points] If you have machines that have cross-datacenter communication across the Wide Area Internet, what check-sum strategy would you use? Justify your answer. [2 points]

Answer – Hardware Software Combined [+1]

Data is passing through a wide area network and that would increase the chances of bit errors. [+1]

c) [2 points] You are now designing a new-generation low-latency RPC system for distributed OS. Your goal is to achieve near-hardware limit RPC performance over modern 400 GBps Ethernet while ensuring strong end-to-end data integrity. Your teammate argues: "Modern NICs and TCP already guarantee reliability. Adding another checksum layer in RPC just increases latency." Do you agree/disagree with this statement? Justify your reasoning. [2 points]

Answer - Agree [+1] No points for disagreement

- TCP protocol inherently provides end-to-end reliability through sequence numbers, acknowledgments, retransmissions, and checksums [+1]

Active Networks [4 points]

1. [4 points] What are two potential security risks when obtaining and storing code in the soft store, and how does the Active Networks Toolkit mitigate these risks?

Ans:

- [+2] Code Spoofing: To combat this, active networks use robust cryptographic fingerprints to verify the integrity of code.
- [+2] Soft State Integrity: Another potential risk is maintaining the integrity of soft state—dynamic data that is not permanently stored. Active networks combat this by enforcing restricted APIs (Application Programming Interfaces) that limit the ways in which the soft state can be modified.
- [+2] Other valid answer

Distributed Objects and Middleware [16 points]

Spring OS [11 points]

1. [3 points]

Your company has two teams developing Spring services that must interoperate. One team is implementing their service in C, while the other team is using C++.

What artifacts must each team produce, so that these services, written in different languages, can communicate seamlessly within the Spring system? Briefly explain how these artifacts will be generated?

Ans:

- [+1] Each team must define interfaces in IDL.
- [+2] Use an IDL compiler to produce:
  - Language specific headers.
  - The client and server stubs.

2. [8 points]

Your coworker is debugging a distributed payment processing service built on Spring OS. Currently, the client process hangs when 'invoice.submit()' is called. The client process runs on Node A, but the invoice service object lives on Node B on a LAN. Your coworker would like to better understand the runtime plumbing so they can trace, reproduce, and fix the bug.

a) [2 points] What is a 'door' and what is a 'network proxy' in this setting, and why are they essential for communication between the client and the invoice service?

Ans:

- [+1] Doors - an abstraction provided by the nucleus to enable secure cross address space object invocation.
- [+1] Network proxies – a mechanism that connects the nuclei of different nodes. Forwards door invocations between the "client" and "invoice service" nodes.

b) (3 points) Explain how this payment service application would have been set up using subcontracts on the spring kernel.

Ans:

Subcontracts would have set up the following;

- Network proxies on Node A and Node B, using the appropriate network protocol between the two proxies.
- Door entry point on node A in the network proxy for the nucleus on node A to receive client request.
- Door call on the network proxy on node B to direct the client call to the door entry point in the payment server.

c) (3 points) At run time, explain the flow of the invocation from the client to the invoice service.

Ans:

- Nucleus on node A will call the door entry point in the network proxy on node A
- network proxy on node A will send the request to network proxy on node B
- network proxy on node B will make a door call to the invoice service entry point through the Nucleus on node B

EJB [2 points]

1. [2 points]

An EJB application built for an internal office network works well. The company now wants to make this service securely available to remote clients over the public Internet (a slow, high-latency WAN).

The service uses fine-grained data objects to get parallelism and performance (like Entity Beans). Explain why a developer would strongly recommend implementing a Session Facade pattern before exposing this service to the wide area network

Ans:

[+1] Each session facade handles business logic associated with its session, and this business logic is not exposed to the insecure public WAN

[+1] This pattern also supports good concurrency since multiple requests can be initiated from the session facade in parallel to multiple entity beans as needed.

Java RMI [3 points]

1. [3 points]

The Java RMI system consists of three main layers: the **stub/skeleton layer**, the **remote reference layer**, and the **transport layer**. For each layer, give one succinct bullet explaining each of these layers.

Ans:

[+1] If the following is conveyed: The stub/skeleton layer is is the interface between the application layer and the rest of the RMI System. Clients stubs and server skeletons will marshal/unmarshal

method arguments and return values to the marshal stream provided by the remote reference layer.

[+1] If the following is conveyed: The remote reference layer handles the semantics of the invocation (e.g. unicast, multicast), and sends data to the connection provided by the transport layer.

[+1] If the following is conveyed: The transport layer handles connection set-up, connection management, and keeping track of and dispatching to remote objects (the targets of remote calls) existing in the transport's local address space.

Distributed Subsystems [24 points]

Global Memory Systems [10 points]

1. [5 points]

Consider a GMS system with 10 Nodes 'A' to 'J'. A process 'x' on Node 'A' intends to read-share a page that is currently actively used only by a process 'y' on Node 'B'. When Node A's GMS mechanism initiates the access, indicate on which Node(s) each of the following data structures are accessed. Assume no co-location constraints for any of the data structures

A) [1 point] POD

Ans: [+1] Node A

B) [1 point] GCD

Ans: [+1] The node that contains the partition for the UID that the page translates to

C) [1 point] PFD

Ans: [+1] Node B

D) [1 point] Indicate the minimum number of network hops needed for the page to be available for use by `x' in the worst-case scenario. (no credit without justification)

Ans: [+1] 3(A -> GCD partition node -> B -> A)

E) [1 point] Now, suppose a new process 'z' spawns on Node 'A' with the intention to read-share the same page. Which Node's PFD shall be accessed this time?

Ans: [+1] Node A

2. [5 points] Consider a 4 Node GMS system (A, B, C, D). Node A needs to evict pages P1 (age=15) and P2 (age=28). In the current epoch, the parameters are as follows: {"minAge":25, "<node:weights>":{"A":0.2, "B":0.3, "C":0.1, "D":0.4}}

a) [2 points] For each page, decide whether Node A should discard locally or send it to a peer node. Provide a brief explanation for each decision.

Ans:

- [+1] P1 -> Send to a peer node (Page is too young to discard; must be preserved somewhere in the cluster to maintain approximate global LRU)
- [+1] P2 -> Discarded locally (Page is old enough; safe to discard)

b) [1 points] If a page has to be sent to a peer node, which is the Node most likely to receive it and why?

Ans:

- [+1] Node D, highest weight

c) (2 points) Mention any additional work that the node A may have to do when the pages are evicted. What is the justification for any such additional work?

Ans:

- (+1) If a page is dirty then it has to be written to the disk before being sent to a peer.
- (+1) Justification: the invariant that needs to be maintained is that all pages in the global port of a node are clean.

Distributed Shared Memory [10 points]

1. [2 points] Consider a DSM application that has no data races. It properly uses synchronization with mutual exclusion locks to safeguard access to shared data structures. The underlying DSM uses SC memory model with page level granularity for coherence. The application experiences poor performance. Explain why.

Ans:

- SC memory model cannot distinguish between synchronization accesses and normal data read/write accesses.
- Every memory access (be it sync variable or normal data) must ensure that the coherence action is fully completed in the entire cluster for the SC memory model.

2. [8 points] In Treadmarks DSM system the following critical section is executed at a node N1:

```
Lock(L1); // Assume X is the only page that been modified up to this
point by prior acquirers of L1
      Write to Page X;
      // Assume that the page is not present at this node;
      // Assume that there are three diff files for page X
      // named Xd2, Xd3, and Xd4 in nodes N2, N3, and N4
      // respectively.
      // Assume the sync causality for the lock L1 is
      // N3 -> N2 -> N1 (i.e, this is the order of lock
      // acquisition).
Unlock (L1);
```

a) [2 points] What actions would be carried out by Treadmarks at the point of lock acquisition? Why?

Ans:

- Page X will be invalidated;
- Why: this is the only page that has been modified by prior acquirers of L1.

b) [2 points] What additional actions would be carried out by Treadmarks at Node 1 before the critical section is executed by N1?

Ans: At the point of access to X, DSM will fetch a clean copy of the page from the owner of that page. Then, it will apply the diffs X_d^3 and X_d^2 in that order to page(X).

c) [2 points] Upon exiting the critical section what action would be carried out by Treadmarks at Node N1?

Ans:

- DSM will record the diff for X (suppose X_d^1) at this node N1
- With the help of OS, it will mark the modified page X as read-only at node N1
- With the help of OS, it will get rid of the twin that was created for page(X).

d) [2 points] A little while later, Node N1 executes the following critical section. Assume that no other node acquired the lock L1 in the interim.

```
Lock(L1);
      Read page X;
      do some computation without changes to any pages;
Unlock(L1);
```

What actions would be carried out by Treadmarks at Node 1 for executing this critical section?

Ans: No action will be taken by DSM since page(X) is already locally available for reading.

Distributed File Systems [4 points]

1. [2 points] In xFS Paper, it's mentioned that metadata Manager Recovery is not Implemented. Propose a high-level algorithm (conceptual level, no need for data structures) for how the xFS system could safely recover from a manager's crash?

Ans:

- [+0.5] Recover the Manager Map using a distributed consensus algorithm.
- [+0.5] Recover Cache Consistency Metadata using "server-driven polling" mechanism
- [+1] Recover Disk Location Metadata using Checkpoint and Roll Forward Mechanism
- [+2] Any Valid algorithm that would allow the system to be resilient to master failure.

2. [2 points]

Background: The Google File System was designed a few years after xFS and **is** heavily influenced by xFS. GFS also separates its metadata management logic from its data storage.

- It uses a **single** "Master" server to store all file system metadata.
- It uses many "Chunkservers" to store actual data blocks.
- A client first contacts the single Master to get metadata and then contacts the Chunkservers directly to get data.

Give one pro and one con of the GFS approach to metadata management.

Ans:

Pro: (any one pro is good enough)

- Simplifies the design since every node knows whom to contact.
- xFS needs additional data structure (mmap) to determine who to contact for every file. This this would be so much more complex if in fact the metadata management is also made dynamic as opposed to static.

Con: (any one con is good enough)

- All file exercise have to go through the central master which could be a performance bottleneck.
- Master failure will bring the system completely down unless care is taken to make sure that master fail over is handled by GFS

HIDDEN - Potpourri [16 points]

1. [2 points] Answer True/False with justification. No credit without justification.

For correctness of a multiprocess application using DSM running on a LAN cluster, a given virtual page should be mapped to the same physical frame number in each node.

Ans:

- No.
- DSM provides address equivalence at virtual page level.

2. [2 points] Answer True/False with justification. No credit without justification.

Multi-writer coherence protocol works with SC, RC and LRC memory models.

Ans: False. Only works with RC and LRC models since they can distinguish between synchronization variable accesses and normal data accesses.

3. [2 points] Answer True/False with justification. No credit without justification.

In Treadmarks, upon a page fault for a page X in Node N1, the DSM software on N1 broadcasts the virtual page number (VPN) of the faulting page to all the peer nodes.

Ans: False. In Treadmarks, there is a manager node statically assigned for every VPN. The DSM software on N1 contacts the manager node for the faulting page X.

4. [4 points] (Answer True/False with justification – No credit without justification)

a) [2 points] In active networks, if the code requested by a node is not present in the previous active node specified in the "prev" field in the capsule header, the request is recursively sent to all the previous active nodes until the code is obtained.

Ans:

- [+2] False, the packet will be dropped if the "prev" node does not have the code. The size of the soft store in a node is limited and is shared by multiple flows. As new packets arrive, code for the old flows has to be replaced, so no previous active nodes will contain the code for this flow.
- [+1] False with incorrect justification

b) [2 points] Active Networks require all intermediate routers between source and destination to be "active", i.e. be able to execute the code in the incoming packet for routing decisions.

Ans:

- [+2] False, if a passive router encounters this packet, it simply reads the IP header and routes the packet. If an active router encounters this packet, it will process the ANTS header and perform given intelligent routing decisions
- [+1] False with incorrect justification

5. [6 points] Your team is experimenting with different RPC communication methods for designing a distributed system. Two design choices are being considered-

- Design A involves LRPC-style mechanism where the client and server communicate through shared memory in an SMP.
- Design B involves traditional RPC where the client and server communicate across the LAN as discussed in the Thekkath & Levy paper.

a) [1 point] For design A, what is the minimum number of context switches required for a single, blocking RPC call-reply cycle?

Answer: 2

b) [1 point] For design A, what are the context switches?

Ans: C-S => one switch from client context switch to server and client blocks after sending the request

S-C => one switch from server to client after it is woken up after receiving the reply

c) [1 point] How can you reduce the number of context switches in Design A even further?

Ans: Having client spin (busy-wait) instead of blocking. [+1]

d) [1 point] For design B, what is the minimum number of context switches required for a single, blocking RPC call-reply cycle?

Ans:

This is a traditional based networking strategy and hence will involve the usual 4

e) [2 points] For design B, what are the context switches?

Ans:

- C-> C1 (client user thread to kernel thread) [0.5]
- S1 ->S (kernel thread on server to user thread on server side) [0.5]
- S -> S2 (user thread on server to kernel thread on server) [0.5]
- C1 -> C (kernel thread on client to user thread on client) [0.5]